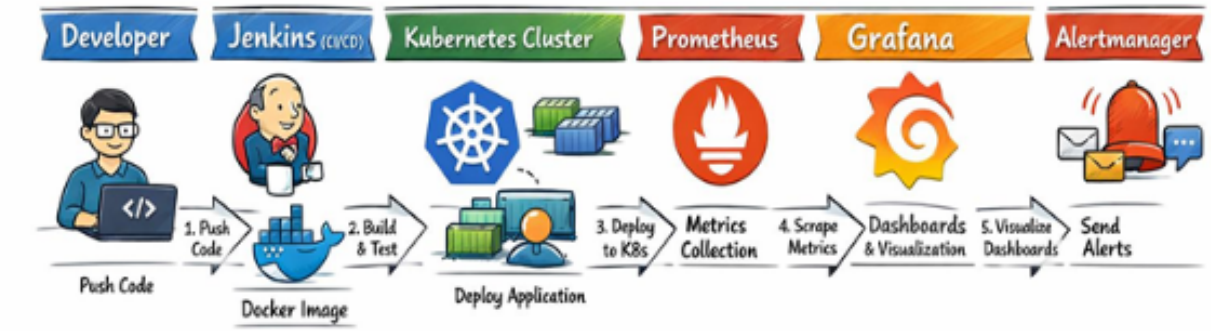
Design and Deployment of an End-to-End DevOps Monitoring SystemUsing AWS, Jenkins, Kubernetes, Prometheus, and Grafana

System Architecture

□ System Architecture



Expected Output

- Live dashboards in Grafana
- Auto-deployment via Jenkins
- Alerts triggered on failures
- Multi-layer monitoring system

Student Name	Aditya Kumar Singh
Roll Number	RA2311003010916

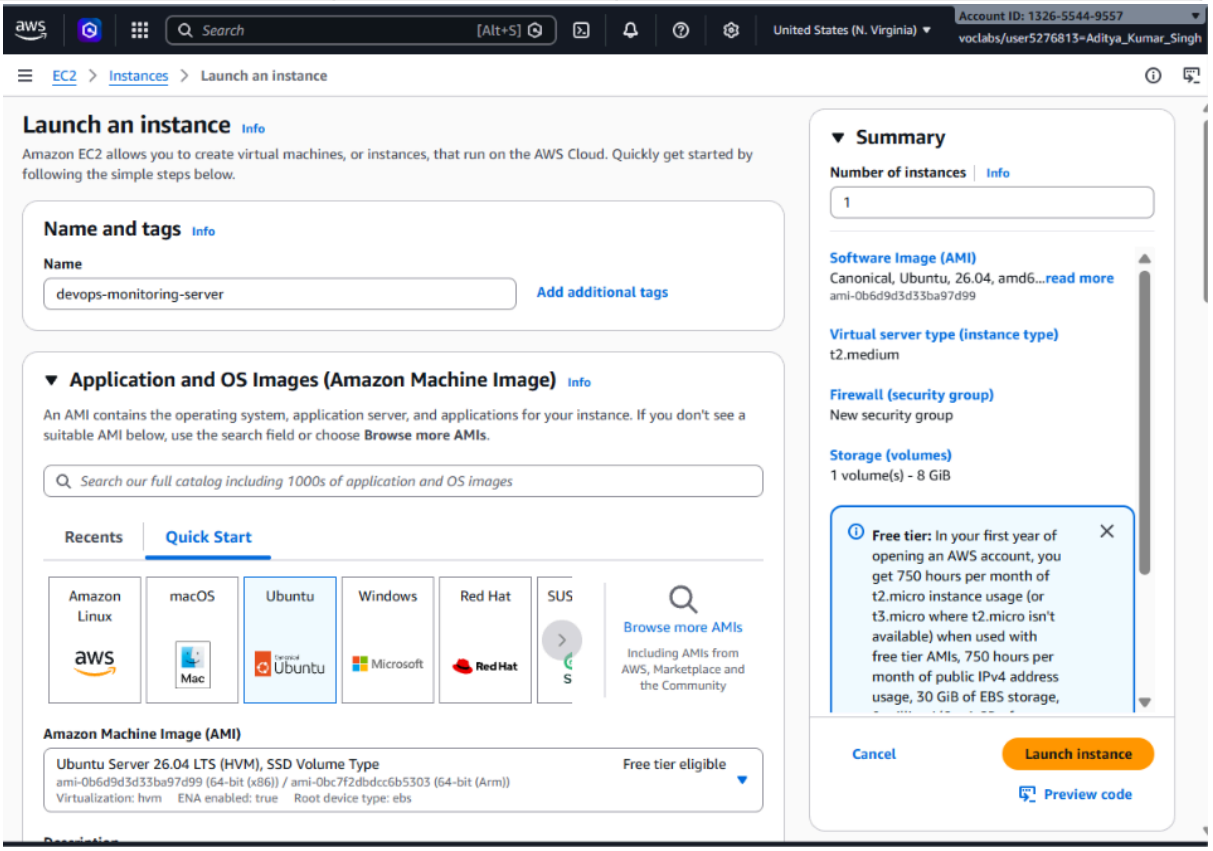
1. Implementation

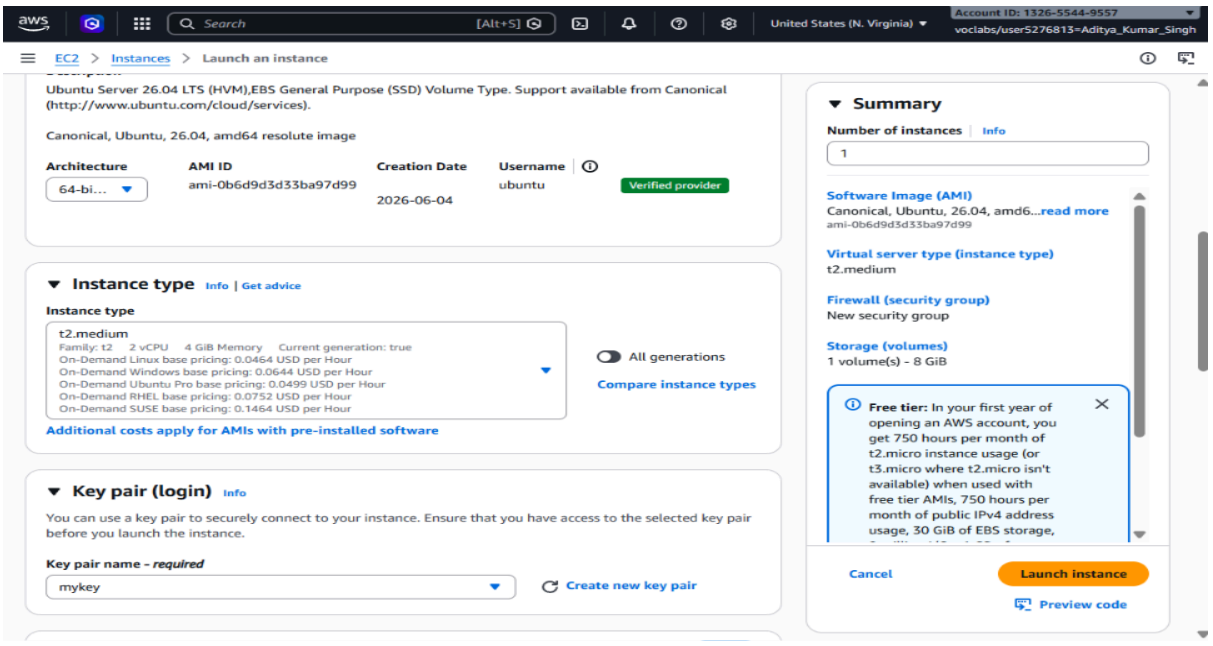
Phase 1 — Infrastructure Setup on AWS

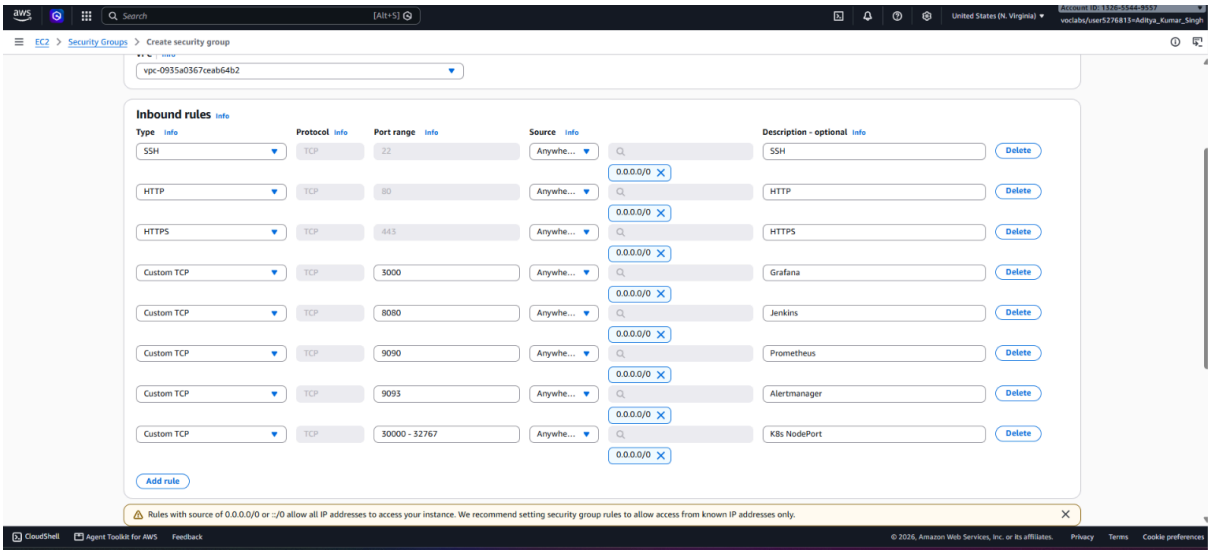
- **Name:** devops-monitoring-server
- **AMI:** Ubuntu Server 22.04 LTS
- **Instance type:** t2.medium (2 vCPU, 4 GB RAM)
- **Key pair:** Create new → download .pem file
- **Storage:** 20 GB gp3
- **Security Group** — add inbound rules:

Table — Security group inbound rules

Port	Protocol	Purpose
22	TCP	SSH administration
8080	TCP	Jenkins web interface
3000	TCP	Grafana dashboards
9090	TCP	Prometheus UI
9093	TCP	Alertmanager UI
30000–32767	TCP	Kubernetes NodePort range



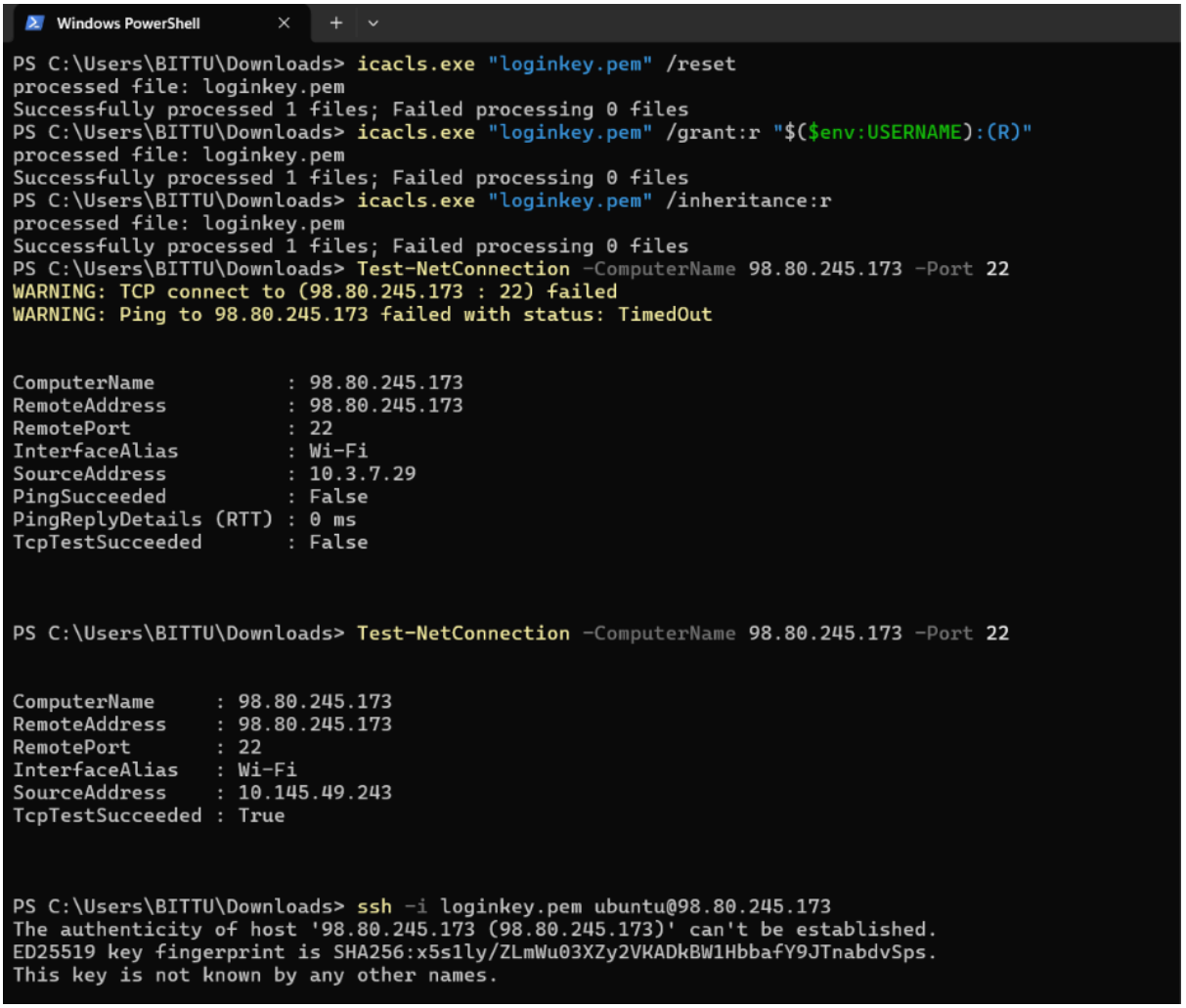


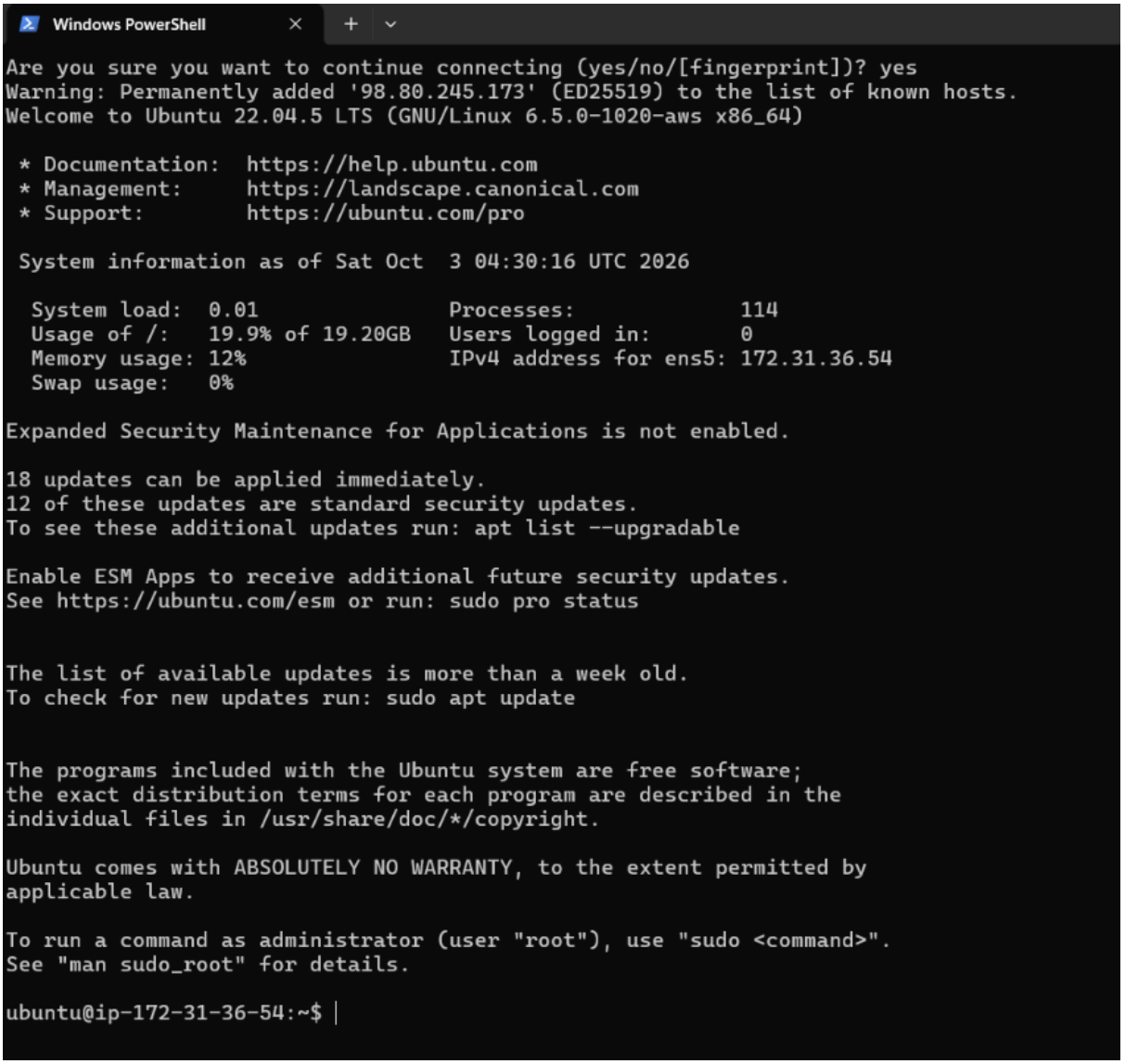


Once the instance reached the Running state, it was accessed from the local machine over SSH:

Launch instance and connect:

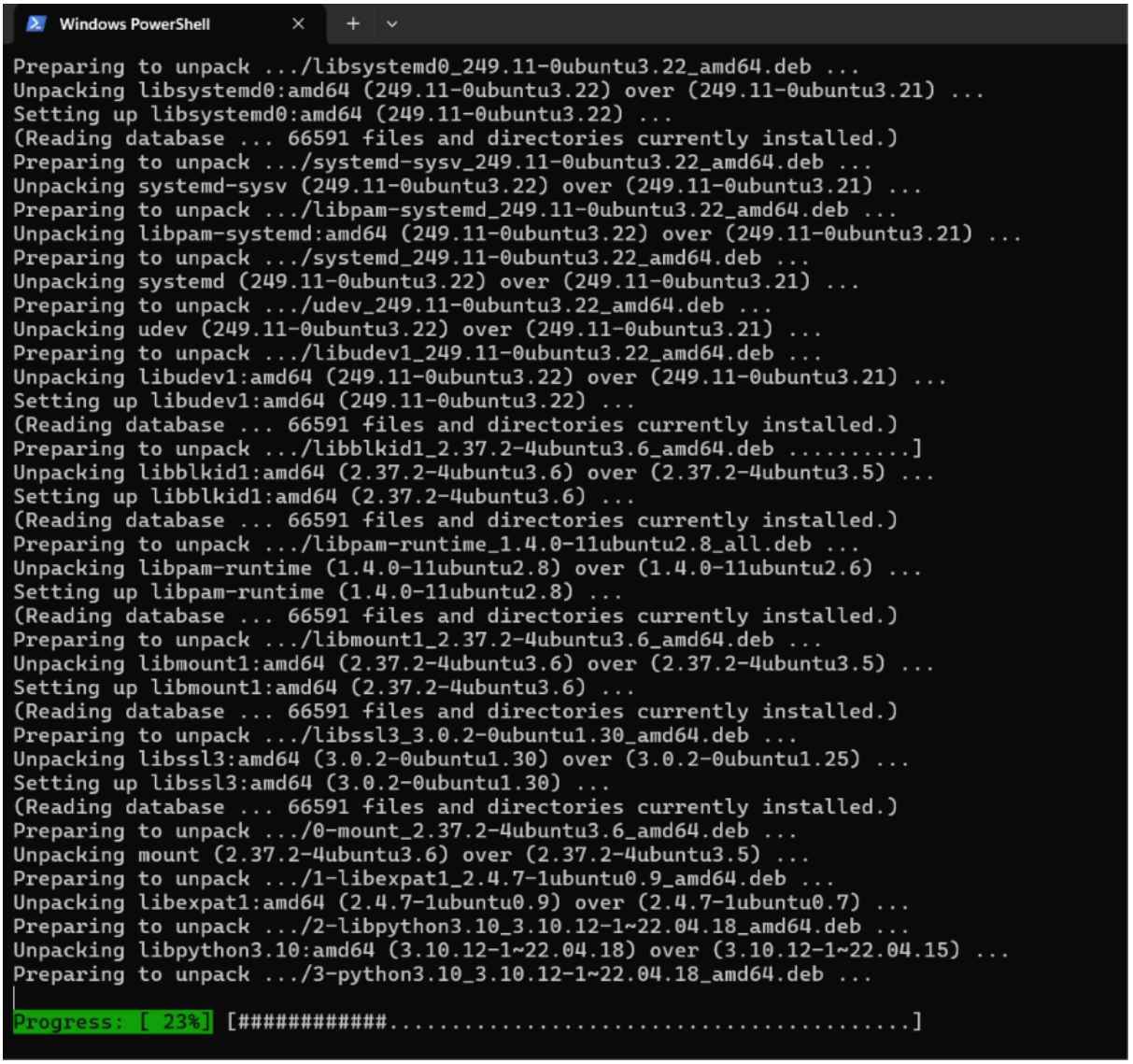
```
chmod 400 devops-key.pem
ssh -i devops-key.pem ubuntu@<EC2_PUBLIC_IP>
```


Windows PowerShell
PS C:\Users\BITTU\Downloads> icacls.exe "loginkey.pem" /reset
processed file: loginkey.pem
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\BITTU\Downloads> icacls.exe "loginkey.pem" /grant:r "$($env:USERNAME):(R)"
processed file: loginkey.pem
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\BITTU\Downloads> icacls.exe "loginkey.pem" /inheritance:r
processed file: loginkey.pem
Successfully processed 1 files; Failed processing 0 files
PS C:\Users\BITTU\Downloads> Test-NetConnection -ComputerName 98.80.245.173 -Port 22
WARNING: TCP connect to (98.80.245.173 : 22) failed
WARNING: Ping to 98.80.245.173 failed with status: TimedOut
ComputerName : 98.80.245.173
RemoteAddress : 98.80.245.173
RemotePort : 22
InterfaceAlias : Wi-Fi
SourceAddress : 10.3.7.29
PingSucceeded : False
PingReplyDetails (RTT) : 0 ms
TcpTestSucceeded : False
PS C:\Users\BITTU\Downloads> Test-NetConnection -ComputerName 98.80.245.173 -Port 22
ComputerName : 98.80.245.173
RemoteAddress : 98.80.245.173
RemotePort : 22
InterfaceAlias : Wi-Fi
SourceAddress : 10.145.49.243
TcpTestSucceeded : True
PS C:\Users\BITTU\Downloads> ssh -i loginkey.pem ubuntu@98.80.245.173
The authenticity of host '98.80.245.173 (98.80.245.173)' can't be established.
ED25519 key fingerprint is SHA256:x5s1ly/ZLmWu03XZy2VKADkBW1HbbafY9JTnabdvSps.
This key is not known by any other names.

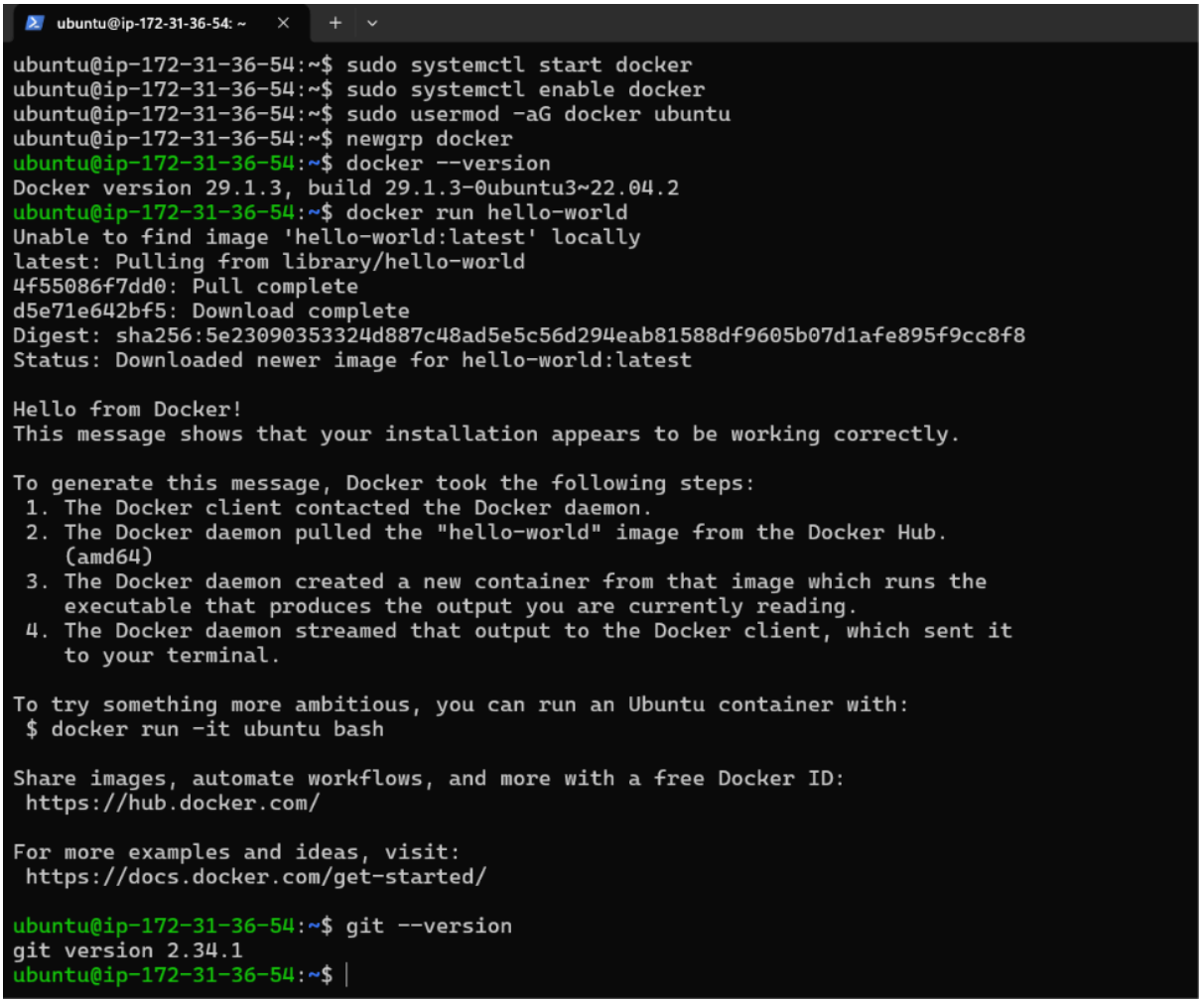


Phase 2 — Base Tools Installation

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl wget unzip apt-transport-https ca-certificates gnupg
```



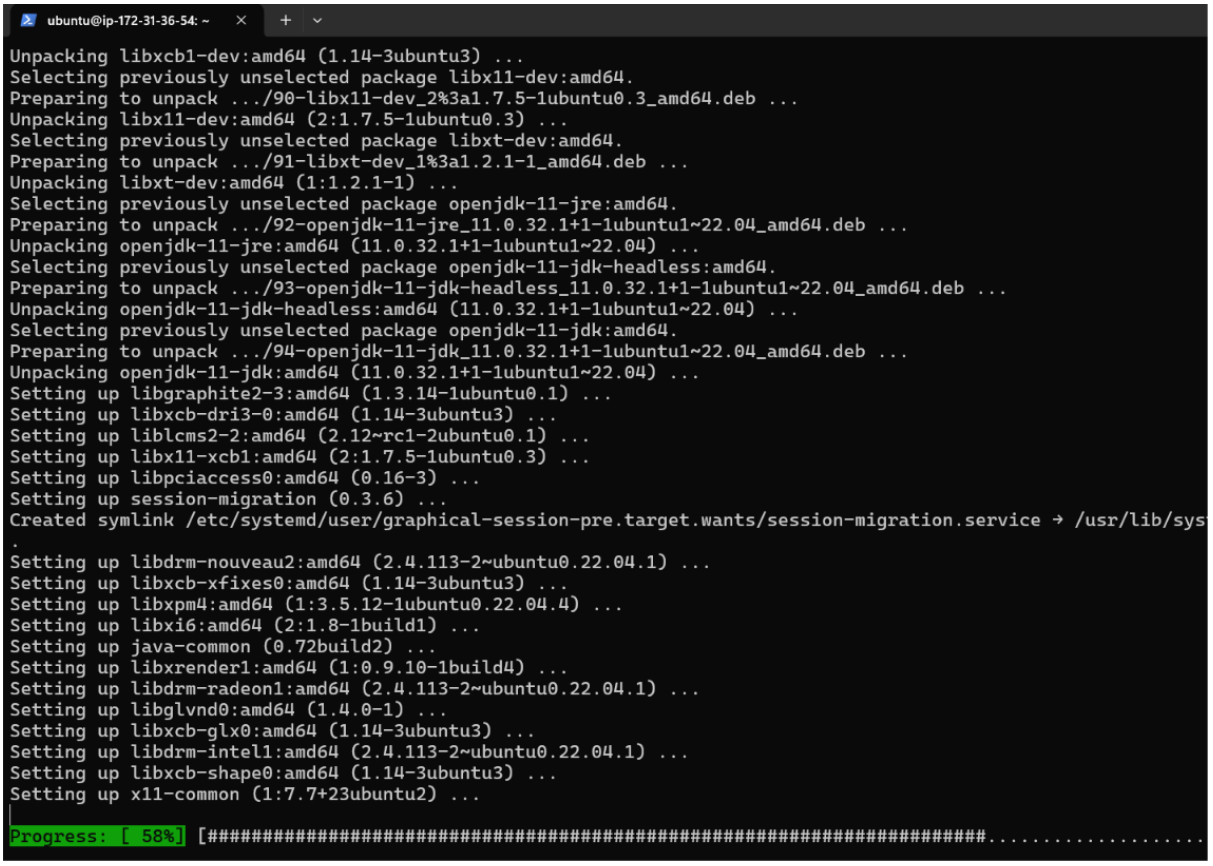

```
sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
newgrp docker
```




```
docker --version
docker run hello-world
```

Phase 3 — Jenkins Installation and Configuration

```
sudo apt install -y fontconfig openjdk-21-jre
java -version
```



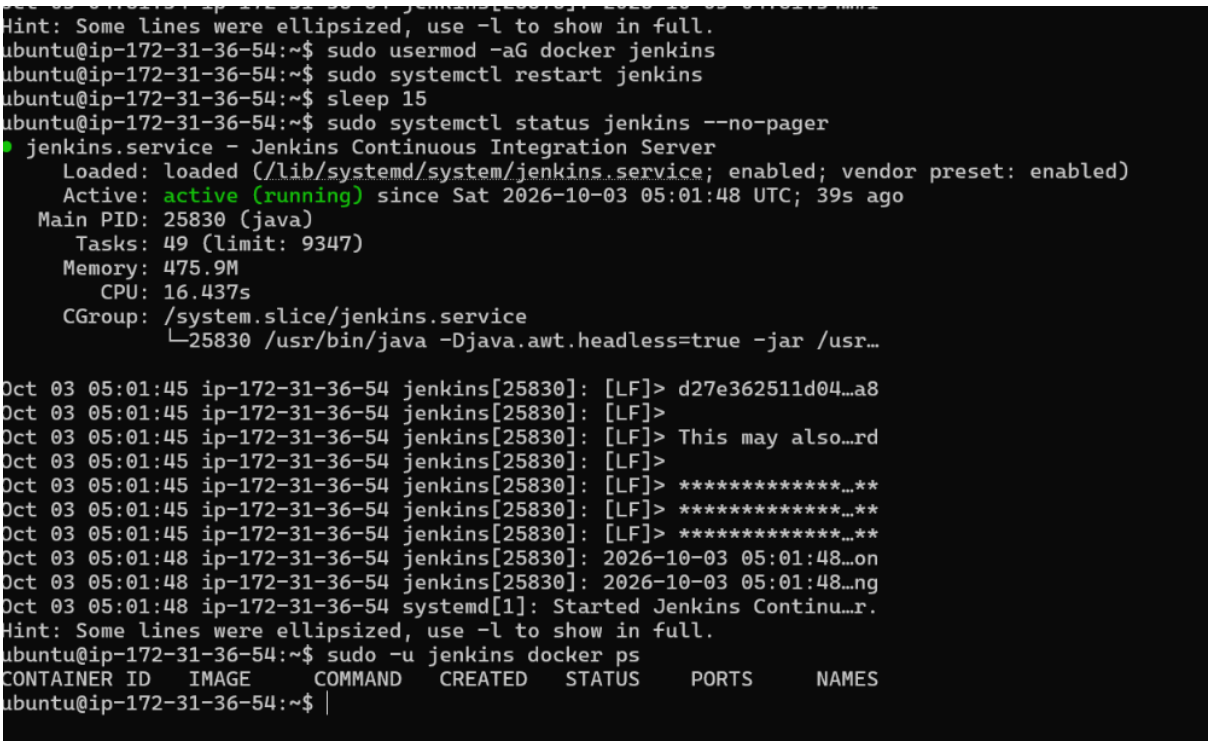



```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install -y jenkins
```

```
ubuntu@ip-172-31-36-54:~$ sudo systemctl daemon-reload
ubuntu@ip-172-31-36-54:~$ sudo systemctl reset-failed jenkins.service
ubuntu@ip-172-31-36-54:~$ sudo systemctl start jenkins
ubuntu@ip-172-31-36-54:~$ sleep 25
ubuntu@ip-172-31-36-54:~$ sudo systemctl status jenkins --no-pager
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/lib/systemd/system/jenkins.service; enabled; ven
dor preset: enabled)
     Active: active (running) since Sat 2026-10-03 04:51:34 UTC; 34s a
go
   Main PID: 25576 (java)
      Tasks: 50 (limit: 9347)
     Memory: 788.8M
        CPU: 19.436s
     CGroup: /system.slice/jenkins.service
             └─25576 /usr/bin/java -Djava.awt.headless=true -jar /usr…

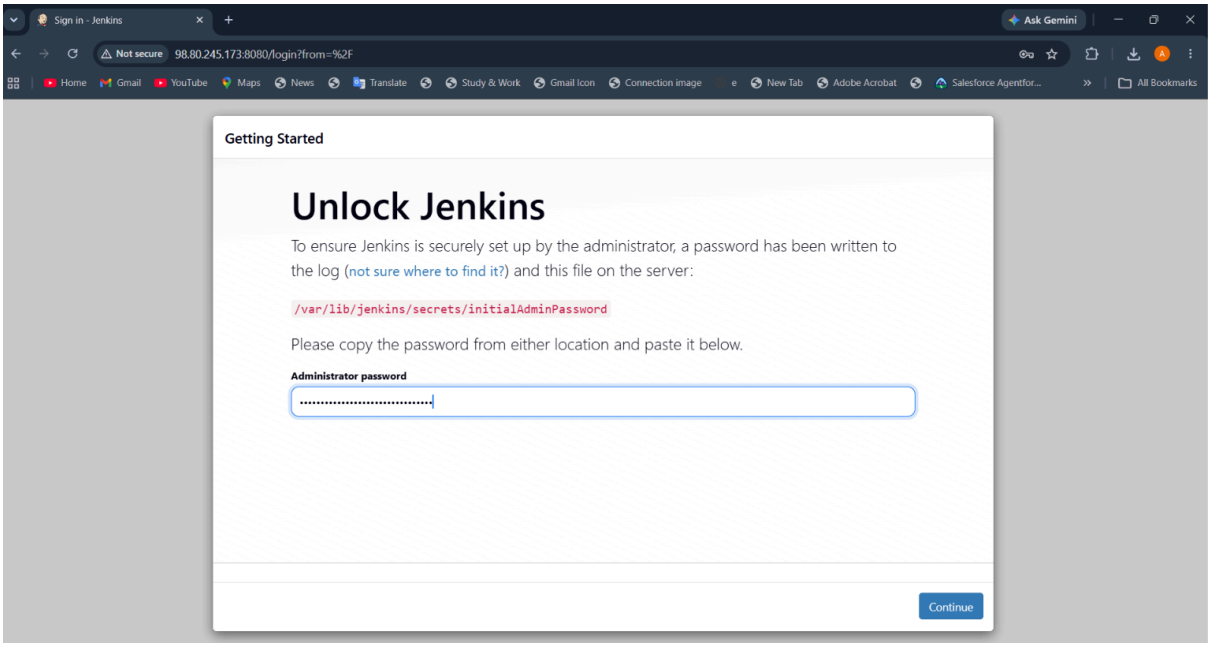
Oct 03 04:51:30 ip-172-31-36-54 jenkins[25576]: [LF]> This may also…rd
Oct 03 04:51:30 ip-172-31-36-54 jenkins[25576]: [LF]>
Oct 03 04:51:30 ip-172-31-36-54 jenkins[25576]: [LF]> **********…**
Oct 03 04:51:30 ip-172-31-36-54 jenkins[25576]: [LF]> **********…**
Oct 03 04:51:30 ip-172-31-36-54 jenkins[25576]: [LF]> **********…**
Oct 03 04:51:34 ip-172-31-36-54 jenkins[25576]: 2026-10-03 04:51:34…on
Oct 03 04:51:34 ip-172-31-36-54 jenkins[25576]: 2026-10-03 04:51:34…ng
Oct 03 04:51:34 ip-172-31-36-54 systemd[1]: Started Jenkins Continu…r.
Oct 03 04:51:34 ip-172-31-36-54 jenkins[25576]: 2026-10-03 04:51:34…er
Oct 03 04:51:34 ip-172-31-36-54 jenkins[25576]: 2026-10-03 04:51:34…#1
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@ip-172-31-36-54:~$ sudo cat /var/lib/jenkins/secrets/initialAdm
inPassword
d27e362511d045d7ab64148417f478a8
ubuntu@ip-172-31-36-54:~$ client_loop: send disconnect: Connection res
et
```

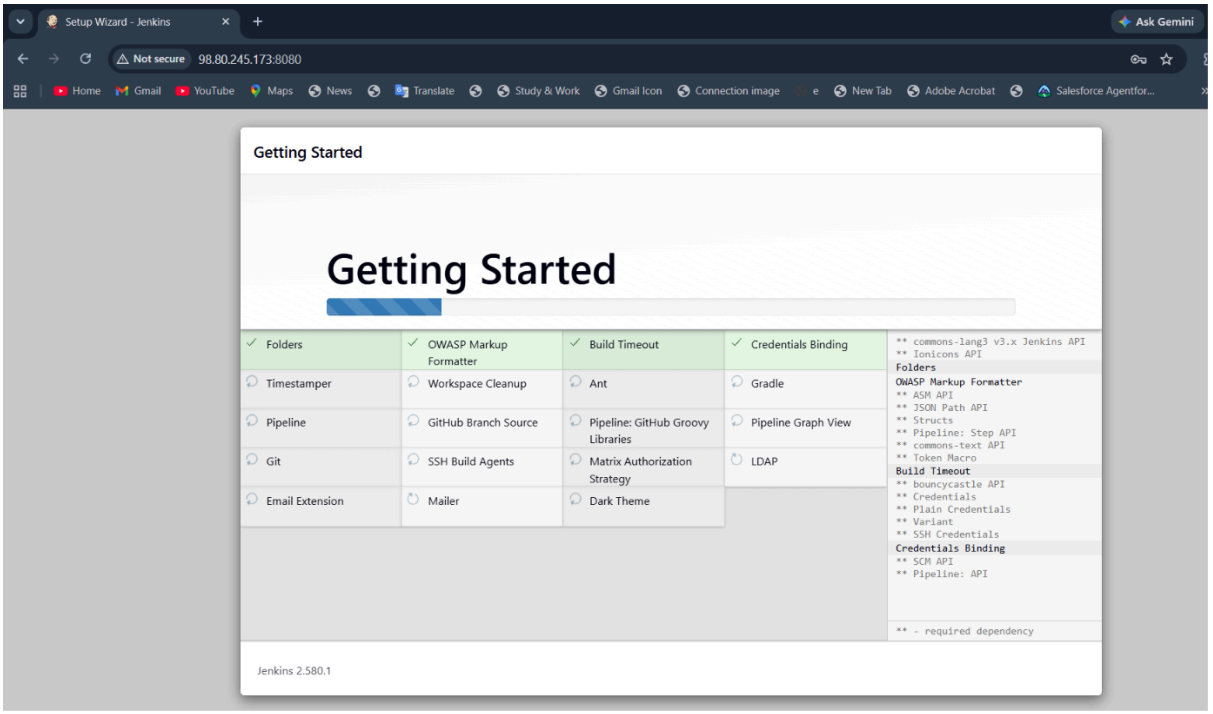
```
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
```

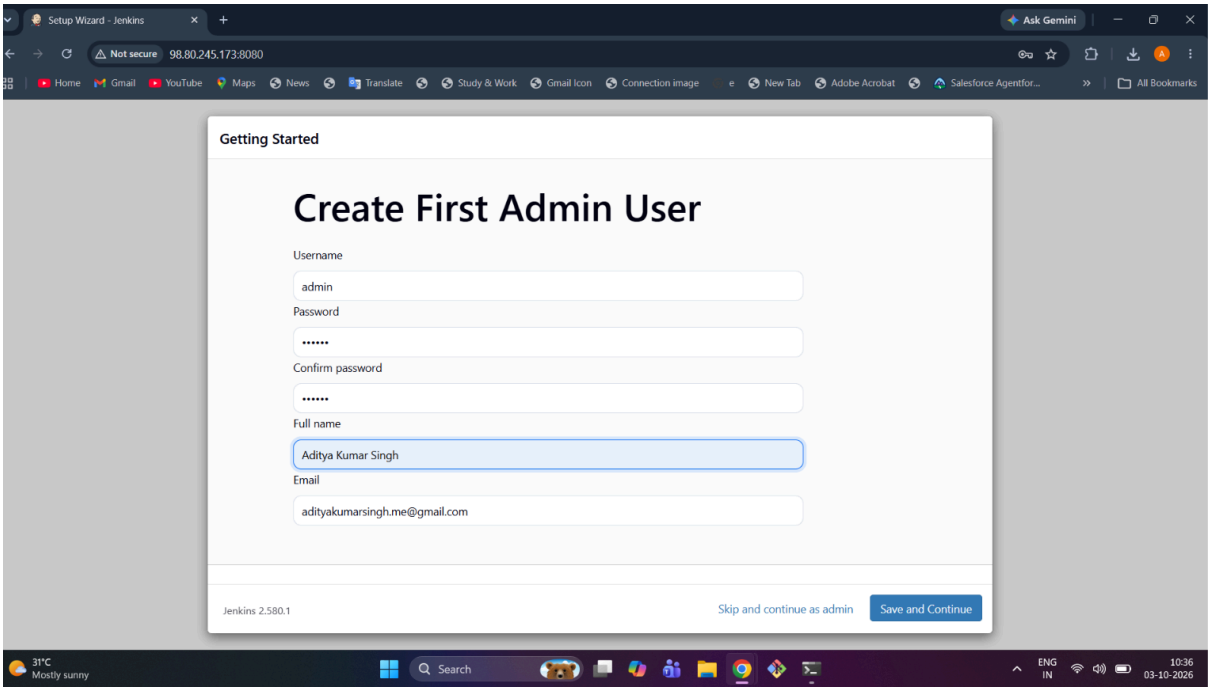



```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins

sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```


Sign in - Jenkins
Not secure
98.80.245.173:8080/login?from=%2F
Getting Started
Unlock Jenkins
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:
/var/lib/jenkins/secrets/initialAdminPassword
Please copy the password from either location and paste it below.
Administrator password
Continue


Setup Wizard - Jenkins
Not secure
98.80.245.173:8080
Getting Started
Getting Started
Folders
OWASP Markup Formatter
Build Timeout
Credentials Binding
Timestamper
Workspace Cleanup
Ant
Gradle
Pipeline
GitHub Branch Source
Pipeline: GitHub Groovy Libraries
Pipeline Graph View
Git
SSH Build Agents
Matrix Authorization Strategy
LDAP
Email Extension
Mailer
Dark Theme
Jenkins 2.580.1



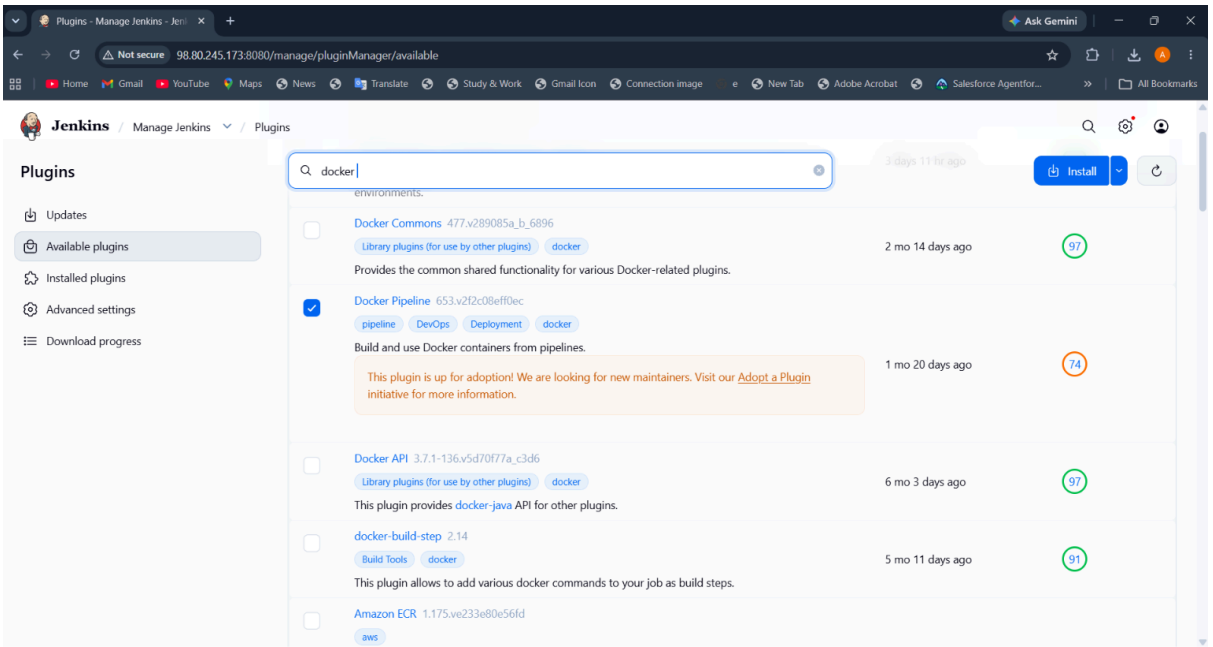
Install Extra Plugins

✅ Docker Pipeline

✅ Git

✅ Kubernetes CLI

✅ Kubernetes



```
ubuntu@ip-172-31-36-54:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~22.04.2
ubuntu@ip-172-31-36-54:~$ git --version
git version 2.34.1
ubuntu@ip-172-31-36-54:~$ java -version
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-22.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-22.04.4-Ubuntu, mixed mo
de, sharing)
ubuntu@ip-172-31-36-54:~$ sudo -u jenkins docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-36-54:~$
```

Phase 4 — Kubernetes Cluster Setup (Minikube)

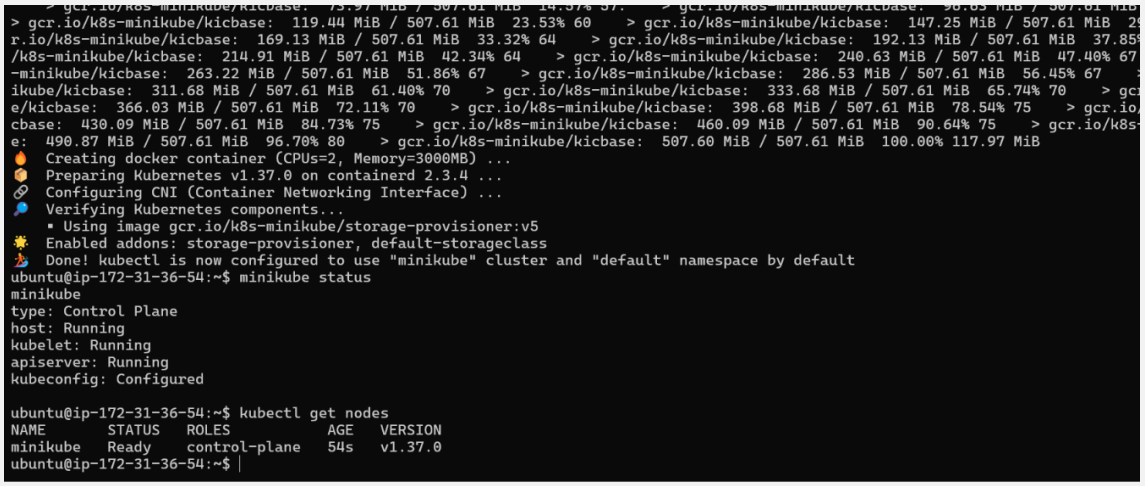
```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

```
ubuntu@ip-172-31-36-54:~$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--100 59.0M  100 59.0M    0     0   235M      0 --:--:-- --:--:-- --:--:--
236M
ubuntu@ip-172-31-36-54:~$ sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
ubuntu@ip-172-31-36-54:~$ kubectl version --client
Client Version: v1.37.1
Kustomize Version: v5.8.1
ubuntu@ip-172-31-36-54:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--100  136M  100  136M    0     0   200M      0 --:--:-- --:--:-- --:--:--
200M
ubuntu@ip-172-31-36-54:~$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
ubuntu@ip-172-31-36-54:~$ minikube version
minikube version: v1.39.0
commit: 7a9f6a841470a207de8cf4bafcccee0969d8ba10
ubuntu@ip-172-31-36-54:~$
```

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube start --driver=docker
```

```
ubuntu@ip-172-31-36-54:~$ minikube start --driver=docker --cpus=2 --memory=3000
😄  minikube v1.39.0 on Ubuntu 22.04
✨  Using the docker driver based on user configuration
📌  Using Docker driver with root privileges
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.51 ...
💾  Downloading Kubernetes v1.37.0 preload ...
    > gcr.io/k8s-minikube/kicbase:  1.60 KiB / 507.61 MiB [>____] 0.00    > preloaded-images-k8s-v18-v1...:  83.65 MiB / 347.58 MiB [] 24.07
> gcr.io/k8s-minikube/kicbase:  1.60 KiB / 507.61 MiB [>____] 0.00    > preloaded-images-k8s-v18-v1...:  138.75 MiB / 347.58 MiB [ 39.92    > gc
r.io/k8s-minikube/kicbase:  15.03 MiB / 507.61 MiB [>___] 2.96    > preloaded-images-k8s-v18-v1...:  193.61 MiB / 347.58 MiB [ 55.70    > gcr.io
/k8s-minikube/kicbase:  34.85 MiB / 507.61 MiB  6.86% 57.2    > preloaded-images-k8s-v18-v1...:  247.92 MiB / 347.58 MiB  71.33%    > gcr.io/k8s
-minikube/kicbase:  55.03 MiB / 507.61 MiB  10.84% 57.    > preloaded-images-k8s-v18-v1...:  300.28 MiB / 347.58 MiB  86.39%    > preloaded-imag
es-k8s-v18-v1...:  347.58 MiB / 347.58 MiB  100.00% 368.48
    > gcr.io/k8s-minikube/kicbase:  73.97 MiB / 507.61 MiB  14.57% 57.    > gcr.io/k8s-minikube/kicbase:  96.63 MiB / 507.61 MiB  19.04% 60.
> gcr.io/k8s-minikube/kicbase:  119.44 MiB / 507.61 MiB  23.53% 60    > gcr.io/k8s-minikube/kicbase:  147.25 MiB / 507.61 MiB  29.01% 60    > gc
r.io/k8s-minikube/kicbase:  169.13 MiB / 507.61 MiB  33.32% 64    > gcr.io/k8s-minikube/kicbase:  192.13 MiB / 507.61 MiB  37.85% 64    > gcr.io
/k8s-minikube/kicbase:  214.91 MiB / 507.61 MiB  42.34% 64    > gcr.io/k8s-minikube/kicbase:  240.63 MiB / 507.61 MiB  47.40% 67    > gcr.io/k8s
-minikube/kicbase:  263.22 MiB / 507.61 MiB  51.86% 67    > gcr.io/k8s-minikube/kicbase:  286.53 MiB / 507.61 MiB  56.45% 67    > gcr.io/k8s-min
ikube/kicbase:  311.68 MiB / 507.61 MiB  61.40% 70    > gcr.io/k8s-minikube/kicbase:  333.68 MiB / 507.61 MiB  65.74% 70    > gcr.io/k8s-minikub
e/kicbase:  366.03 MiB / 507.61 MiB  72.11% 70    > gcr.io/k8s-minikube/kicbase:  398.68 MiB / 507.61 MiB  78.54% 75    > gcr.io/k8s-minikube/ki
cbase:  430.09 MiB / 507.61 MiB  84.73% 75    > gcr.io/k8s-minikube/kicbase:  460.09 MiB / 507.61 MiB  90.64% 75    > gcr.io/k8s-minikube/kicbas
e:  490.87 MiB / 507.61 MiB  96.70% 80    > gcr.io/k8s-minikube/kicbase:  507.60 MiB / 507.61 MiB  100.00% 117.97 MiB
🔥  Creating docker container (CPUs=2, Memory=3000MB) ...-
```

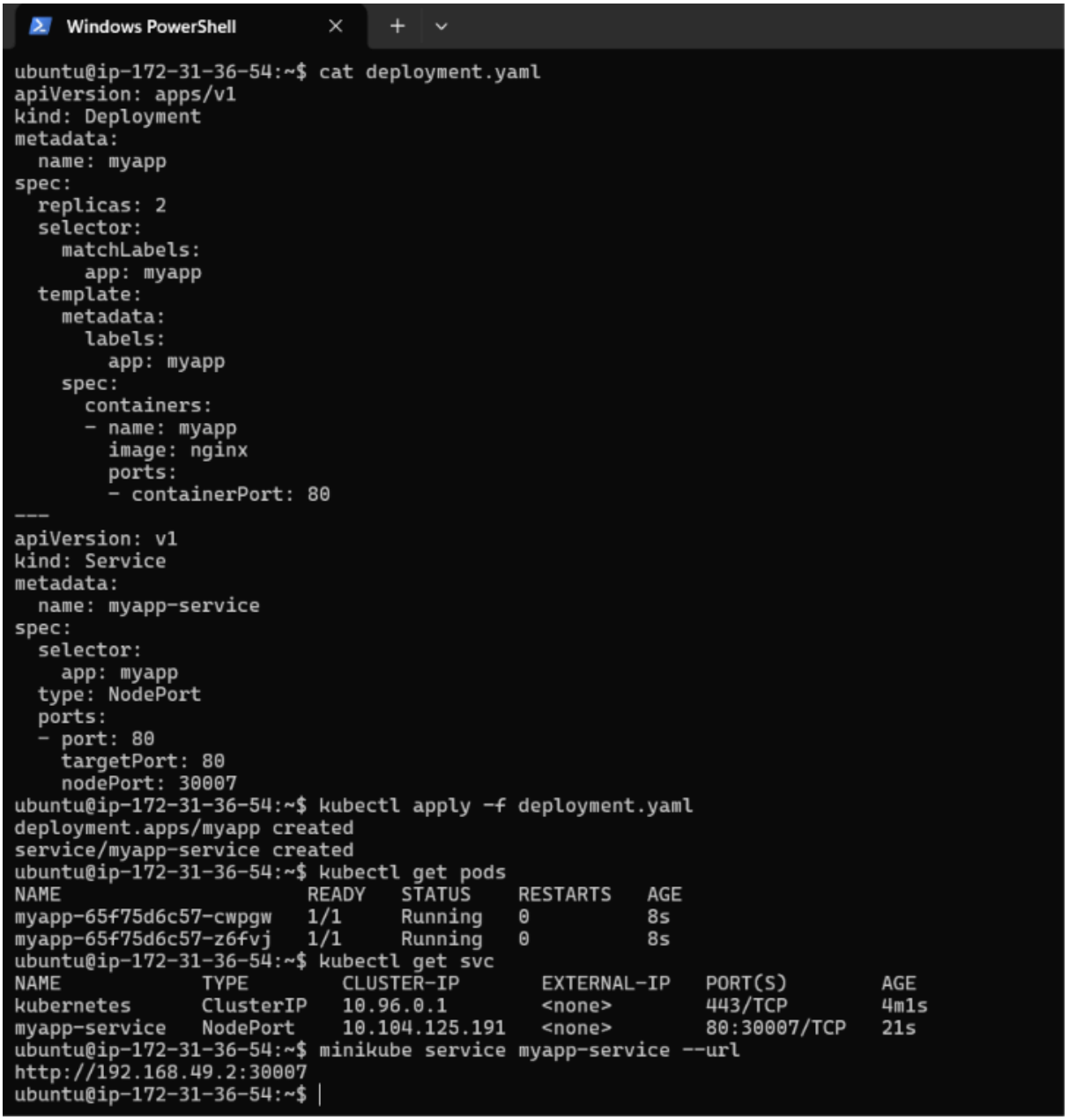
```
kubectl get nodes
minikube status
```



[FIGURE 7: Minikube Cluster Running]

Phase 5 — Sample Application Deployment

```
kubectl apply -f deployment.yaml
kubectl get pods
kubectl get svc
```




```
curl http://$(minikube ip):30007
```

```
ubuntu@ip-172-31-36-54:~$ curl http://192.168.49.2:30007
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Caption: Figure 8 — NGINX application deployed with 2 replicas.

4.6 Phase 6 — Prometheus and Grafana Monitoring Stack

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

```
ubuntu@ip-172-31-36-54:~$ sudo snap install helm --classic
helm 4.3.0 from Snapcrafters✪ installed
ubuntu@ip-172-31-36-54:~$ helm version
version.BuildInfo{Version:"v4.3.0", GitCommit:"bec5b06ed841fe5269972d864d51779044fd5970f", GitTreeState:"clean", GoVersion:"go1.27.1", KubeClientVersion:"v1.37"}
ubuntu@ip-172-31-36-54:~$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
ubuntu@ip-172-31-36-54:~$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. ⎈Happy Helming!⎈
ubuntu@ip-172-31-36-54:~$ kubectl create namespace monitoring
namespace/monitoring created
ubuntu@ip-172-31-36-54:~$ helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring
NAME: monitoring
LAST DEPLOYED: Sat Oct  3 05:28:05 2026
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace monitoring get pods -l "release=monitoring"

Get Grafana 'admin' user password by running:

  kubectl --namespace monitoring get secrets monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo

Access Grafana local instance:

  export POD_NAME=$(kubectl --namespace monitoring get pod -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=monitoring" -oname)
  kubectl --namespace monitoring port-forward $POD_NAME 3000

Get your grafana admin user password by running:

  kubectl get secret --namespace monitoring -l app.kubernetes.io/component=admin-secret -o jsonpath="{.items[0].data.admin-password}" | base64 --decode ; echo


Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheus instances using the Operator.
ubuntu@ip-172-31-36-54:~$
```

```
kubectl create namespace monitoring
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring
kubectl get pods -n monitoring
```

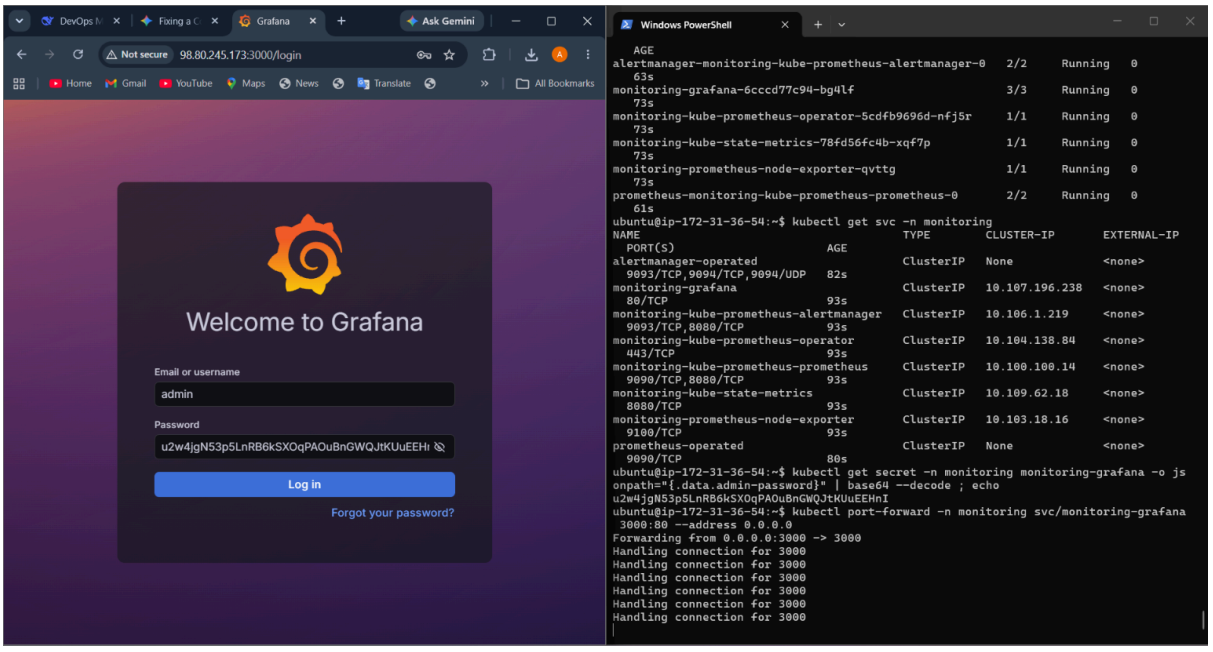
```
ubuntu@ip-172-31-36-54:~$ kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2     Running   0          63s
monitoring-grafana-6cccd77c94-bg4lf                      3/3     Running   0          73s
monitoring-kube-prometheus-operator-5cdfb9696d-nfj5r     1/1     Running   0          73s
monitoring-kube-state-metrics-78fd56fc4b-xqf7p           1/1     Running   0          73s
monitoring-prometheus-node-exporter-qvttg                1/1     Running   0          73s
prometheus-monitoring-kube-prometheus-prometheus-0       2/2     Running   0          61s
ubuntu@ip-172-31-36-54:~$ kubectl get svc -n monitoring
NAME                                      TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
alertmanager-operated                     ClusterIP   None             <none>        9093/TCP,9094/TCP,9094/UDP   82s
monitoring-grafana                        ClusterIP   10.107.196.238   <none>        80/TCP                       93s
monitoring-kube-prometheus-alertmanager   ClusterIP   10.106.1.219     <none>        9093/TCP,8080/TCP            93s
monitoring-kube-prometheus-operator       ClusterIP   10.104.138.84    <none>        443/TCP                      93s
monitoring-kube-prometheus-prometheus     ClusterIP   10.100.100.14    <none>        9090/TCP,8080/TCP            93s
monitoring-kube-state-metrics             ClusterIP   10.109.62.18     <none>        8080/TCP                     93s
monitoring-prometheus-node-exporter       ClusterIP   10.103.18.16     <none>        9100/TCP                     93s
prometheus-operated                       ClusterIP   None             <none>        9090/TCP                     80s
ubuntu@ip-172-31-36-54:~$
```

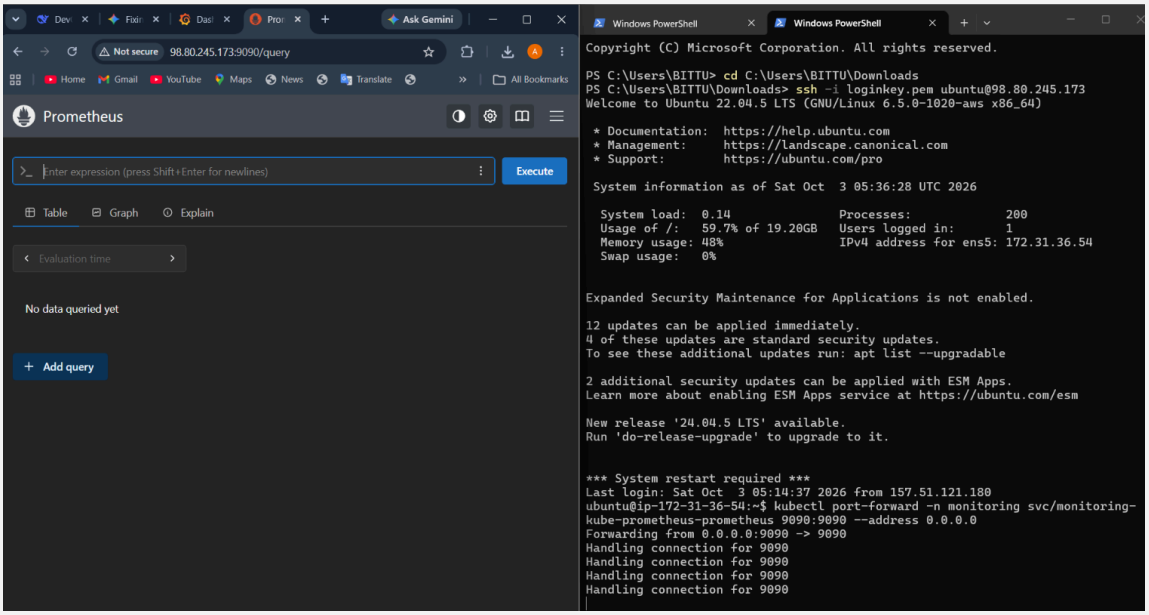
Caption: Figure 9 — Full monitoring stack running in the monitoring namespace.

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80 --address 0.0.0.0 &
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090
--address 0.0.0.0 &
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-alertmanager 9093:9093
--address 0.0.0.0 &
```

The Grafana administrator password, which is stored in a Kubernetes secret, was retrieved with:

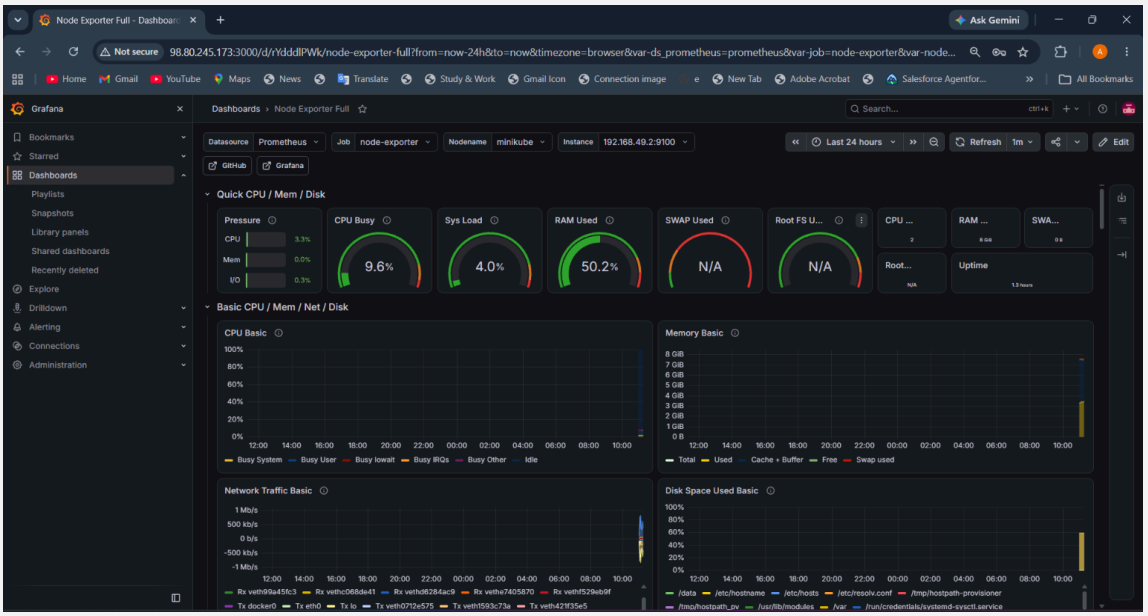
```
kubectl get secret -n monitoring monitoring-grafana \
  -o jsonpath="{.data.admin-password}" | base64 --decode; echo
```





Caption: Figure 10 — Prometheus scraping all cluster endpoints successfully.

Grafana, available at `http://<EC2_PUBLIC_IP>:3000`, was then logged into with the `admin` user. The chart provisions Prometheus as a data source automatically. Two community dashboards were additionally imported by ID through **Dashboards → Import**. The first, **Node Exporter Full (ID 1860)**, displays host-level CPU, memory, disk, and network metrics in detail.



Caption: Figure 11 — Grafana Node Exporter dashboard with live metrics.

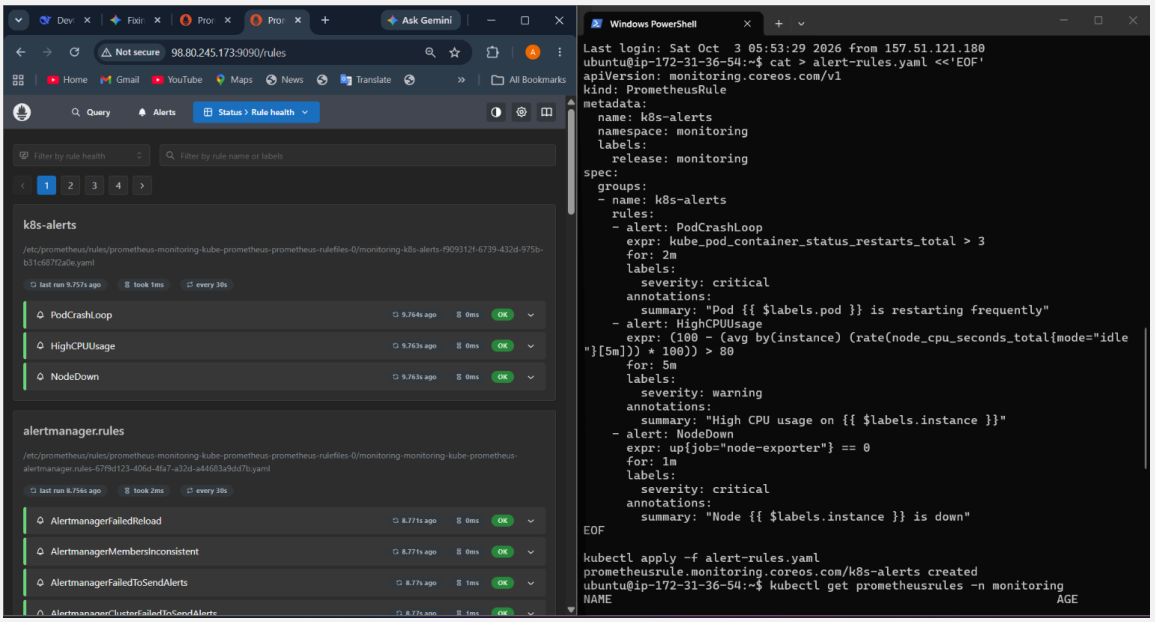
```
ubuntu@ip-172-31-36-54:~$ sudo ss -tlnp | grep -E ":3000|:9090|:9093" || ech
o "All ports free"
All ports free
ubuntu@ip-172-31-36-54:~$ nohup kubectl port-forward -n monitoring svc/monit
oring-grafana 3000:80 --address 0.0.0.0 > /tmp/grafana-pf.log 2>&1 &
[1] 32670
ubuntu@ip-172-31-36-54:~$ nohup kubectl port-forward -n monitoring svc/monit
oring-kube-prometheus-prometheus 9090:9090 --address 0.0.0.0 > /tmp/promethe
us-pf.log 2>&1 &
[2] 32677
ubuntu@ip-172-31-36-54:~$ nohup kubectl port-forward -n monitoring svc/monit
oring-kube-prometheus-alertmanager 9093:9093 --address 0.0.0.0 > /tmp/alertm
anager-pf.log 2>&1 &
[3] 32689
ubuntu@ip-172-31-36-54:~$ echo "=== Grafana ===" && cat /tmp/grafana-pf.log
=== Grafana ===
nohup: ignoring input
Forwarding from 0.0.0.0:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
ubuntu@ip-172-31-36-54:~$ echo "=== Prometheus ===" && cat /tmp/prometheus-p
f.log
=== Prometheus ===
nohup: ignoring input
Forwarding from 0.0.0.0:9090 -> 9090
ubuntu@ip-172-31-36-54:~$ echo "=== Alertmanager ===" && cat /tmp/alertmanag
er-pf.log
=== Alertmanager ===
nohup: ignoring input
Forwarding from 0.0.0.0:9093 -> 9093
ubuntu@ip-172-31-36-54:~$
```

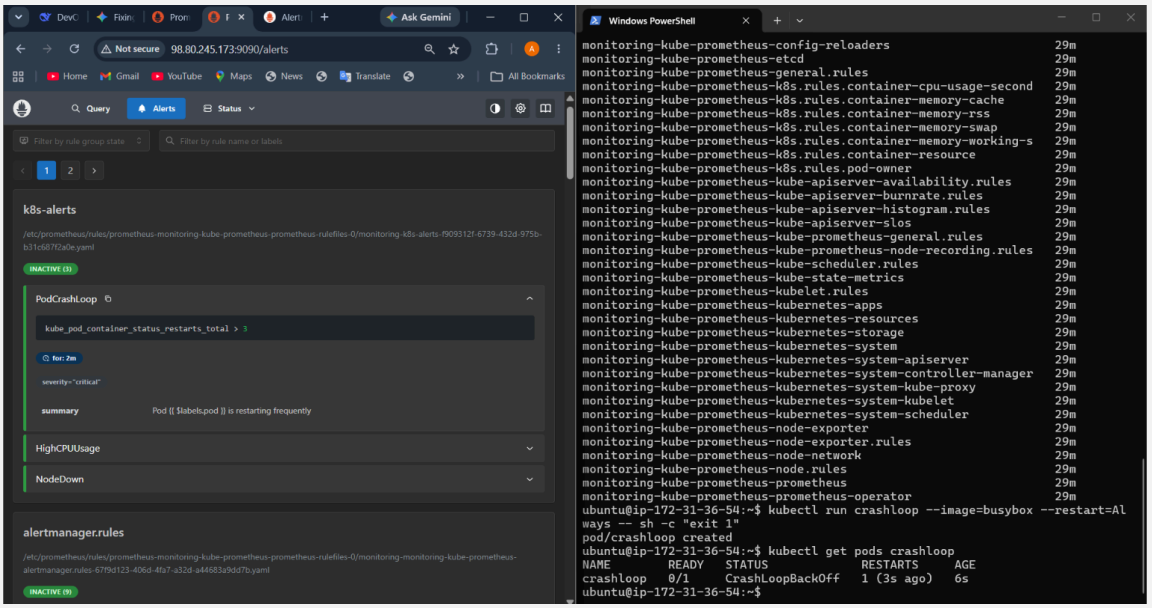
Phase 7 — Alerting with PrometheusRule and Alertmanager

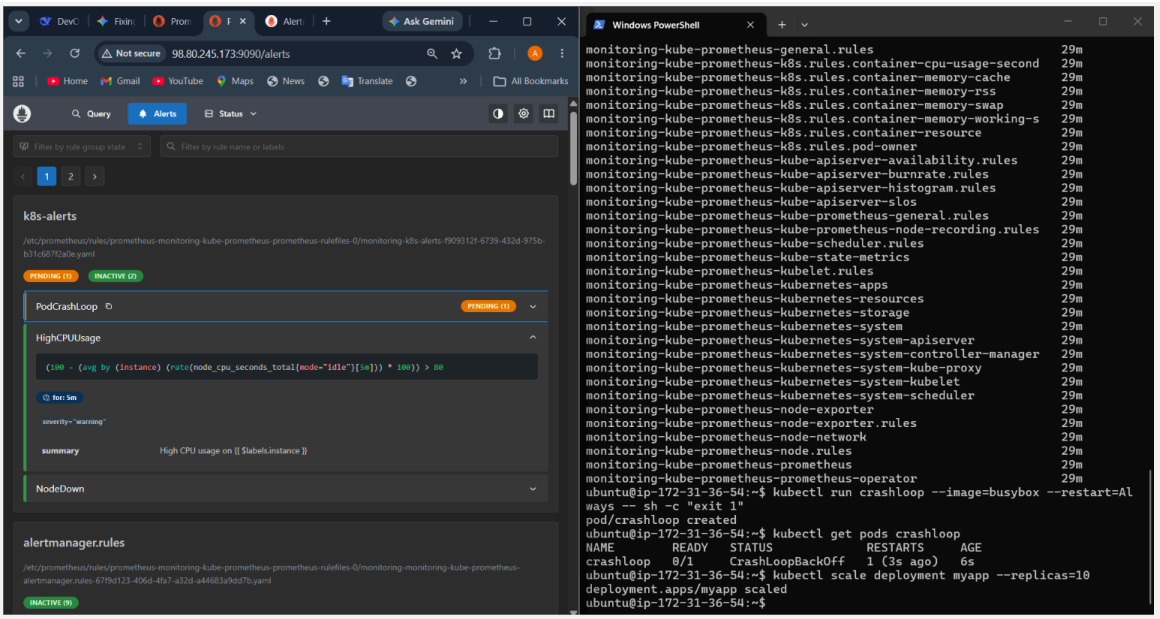
Table 5 — Custom alert rules

Alert	Condition	Severity	Purpose
PodCrashLoop	Container restart count increases repeatedly within a time window	critical	Detects pods that repeatedly fail and restart
HighCPUUsage	Node CPU utilization above a threshold (80%) for 2 minutes	warning	Detects sustained resource pressure
NodeDown	A node's up metric equal to 0 for 1 minute	critical	Detects loss of a monitored node

```
kubectl apply -f alert-rules.yaml
kubectl get prometheusrule -n monitoring
```

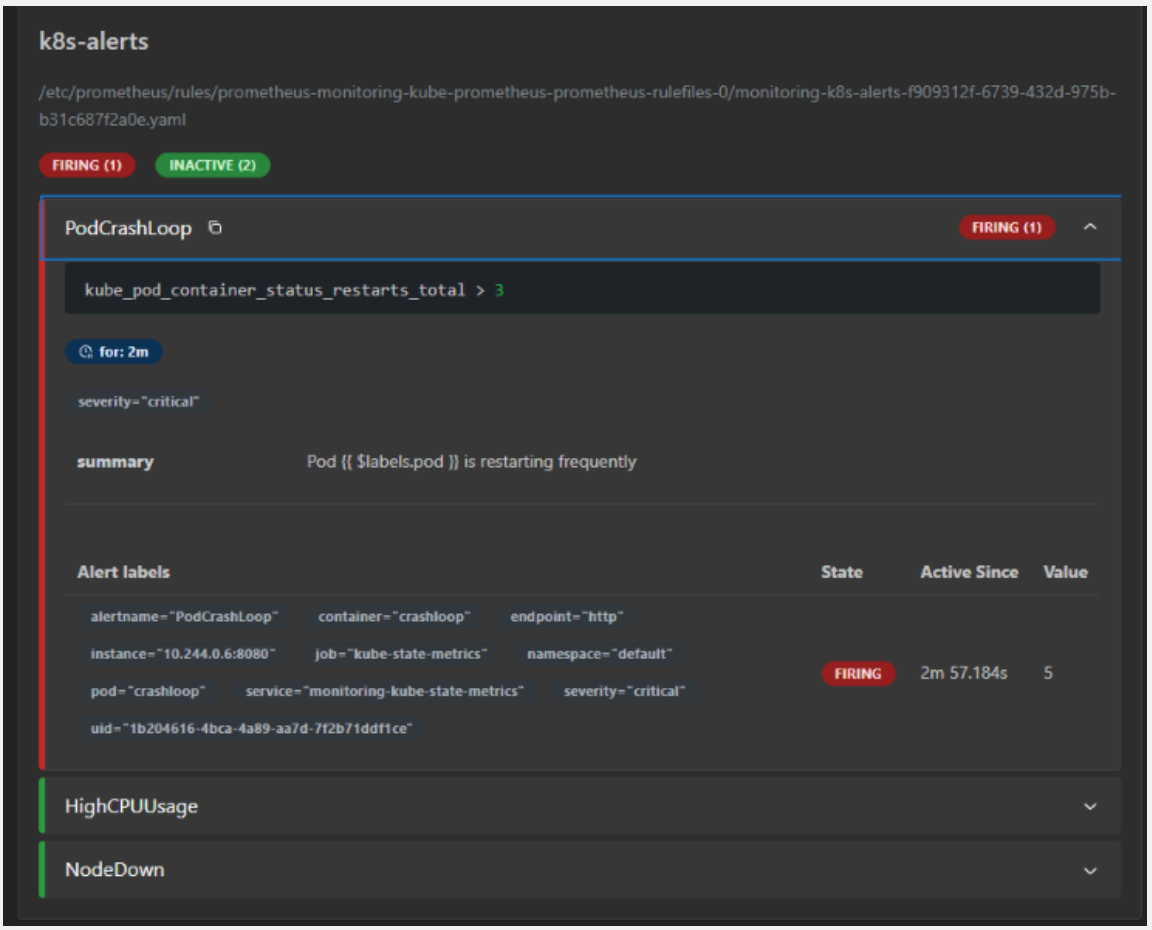

Ask Gemini
Not secure 98.80.245.173:9090/rules
Home Gmail YouTube Maps News Translate All Bookmarks
Query Alerts Status > Rule health
k8s-alerts
/etc/prometheus/rules/prometheus-monitoring-kube-prometheus-prometheus-rulefiles-0/monitoring-k8s-alerts-f909312f-6739-432d-975b-b31c687f2a0e.yaml
PodCrashLoop
HighCPUUsage
NodeDown
alertmanager.rules
/etc/prometheus/rules/prometheus-monitoring-kube-prometheus-prometheus-rulefiles-0/monitoring-monitoring-kube-prometheus-alertmanager.rules-67f9d123-406d-4fa7-a32d-a44683a9dd7b.yaml
AlertmanagerFailedReload
AlertmanagerMembersInconsistent
AlertmanagerFailedToSendAlerts
Windows PowerShell
Last login: Sat Oct 3 05:53:29 2026 from 157.51.121.180
ubuntu@ip-172-31-36-54:~$ cat > alert-rules.yaml <<'EOF'
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
name: k8s-alerts
namespace: monitoring
labels:
release: monitoring
spec:
groups:
- name: k8s-alerts
rules:
- alert: PodCrashLoop
expr: kube_pod_container_status_restarts_total > 3
for: 2m
labels:
severity: critical
annotations:
summary: "Pod {{ $labels.pod }} is restarting frequently"
- alert: HighCPUUsage
expr: (100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)) > 80
for: 5m
labels:
severity: warning
annotations:
summary: "High CPU usage on {{ $labels.instance }}"
- alert: NodeDown
expr: up{job="node-exporter"} == 0
for: 1m
labels:
severity: critical
annotations:
summary: "Node {{ $labels.instance }} is down"
EOF
kubectl apply -f alert-rules.yaml
prometheusrule.monitoring.coreos.com/k8s-alerts created
ubuntu@ip-172-31-36-54:~$ kubectl get prometheusrules -n monitoring
NAME AGE


Ask Gemini
Not secure 98.80.245.173:9090/alerts
Home Gmail YouTube Maps News Translate All Bookmarks
Query Alerts Status
k8s-alerts
/etc/prometheus/rules/prometheus-monitoring-kube-prometheus-prometheus-rulefiles-0/monitoring-k8s-alerts-f909312f-6739-432d-975b-b31c687f2a0e.yaml
INACTIVE (3)
PodCrashLoop
kube_pod_container_status_restarts_total > 3
for: 2m
severity="critical"
summary Pod {{ $labels.pod }} is restarting frequently
HighCPUUsage
NodeDown
alertmanager.rules
/etc/prometheus/rules/prometheus-monitoring-kube-prometheus-prometheus-rulefiles-0/monitoring-monitoring-kube-prometheus-alertmanager.rules-67f9d123-406d-4fa7-a32d-a44683a9dd7b.yaml
INACTIVE (9)
Windows PowerShell
monitoring-kube-prometheus-config-reloaders 29m
monitoring-kube-prometheus-etcd 29m
monitoring-kube-prometheus-general.rules 29m
monitoring-kube-prometheus-k8s.rules.container-cpu-usage-second 29m
monitoring-kube-prometheus-k8s.rules.container-memory-cache 29m
monitoring-kube-prometheus-k8s.rules.container-memory-rss 29m
monitoring-kube-prometheus-k8s.rules.container-memory-swap 29m
monitoring-kube-prometheus-k8s.rules.container-memory-working-s 29m
monitoring-kube-prometheus-k8s.rules.container-resource 29m
monitoring-kube-prometheus-k8s.rules.pod-owner 29m
monitoring-kube-prometheus-kube-apiserver-availability.rules 29m
monitoring-kube-prometheus-kube-apiserver-burnrate.rules 29m
monitoring-kube-prometheus-kube-apiserver-histogram.rules 29m
monitoring-kube-prometheus-kube-apiserver-slos 29m
monitoring-kube-prometheus-kube-prometheus-general.rules 29m
monitoring-kube-prometheus-kube-prometheus-node-recording.rules 29m
monitoring-kube-prometheus-kube-scheduler.rules 29m
monitoring-kube-prometheus-kube-state-metrics 29m
monitoring-kube-prometheus-kubelet.rules 29m
monitoring-kube-prometheus-kubernetes-apps 29m
monitoring-kube-prometheus-kubernetes-resources 29m
monitoring-kube-prometheus-kubernetes-storage 29m
monitoring-kube-prometheus-kubernetes-system 29m
monitoring-kube-prometheus-kubernetes-system-apiserver 29m
monitoring-kube-prometheus-kubernetes-system-controller-manager 29m
monitoring-kube-prometheus-kubernetes-system-kube-proxy 29m
monitoring-kube-prometheus-kubernetes-system-kubelet 29m
monitoring-kube-prometheus-kubernetes-system-scheduler 29m
monitoring-kube-prometheus-node-exporter 29m
monitoring-kube-prometheus-node-exporter.rules 29m
monitoring-kube-prometheus-node-network 29m
monitoring-kube-prometheus-node.rules 29m
monitoring-kube-prometheus-prometheus 29m
monitoring-kube-prometheus-prometheus-operator 29m
ubuntu@ip-172-31-36-54:~$ kubectl run crashloop --image=busybox --restart=Always -- sh -c "exit 1"
pod/crashloop created
ubuntu@ip-172-31-36-54:~$ kubectl get pods crashloop
NAME READY STATUS RESTARTS AGE
crashloop 0/1 CrashLoopBackOff 1 (3s ago) 6s
ubuntu@ip-172-31-36-54:~$



Caption: Figure 13 — Custom alert rules registered in Prometheus.

```
kubectl run crash-test --image=busybox --restart=Always -- /bin/sh -c "exit 1"
kubectl get pods
```



Caption: Figure 14 — PodCrashLoop alert firing after simulated pod failure.

Appendix

A. Full Jenkinsfile

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "myapp"
        IMAGE_TAG  = "v${BUILD_NUMBER}"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                sh 'docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .'
            }
        }

        stage('Load into Minikube') {
            steps {
                sh 'minikube image load ${IMAGE_NAME}:${IMAGE_TAG}'
            }
        }

        stage('Deploy') {
            steps {
                sh '''
                    sed -i "s|image: myapp:.*|image: ${IMAGE_NAME}:${IMAGE_TAG}|"
deployment.yaml
                    kubectl apply -f deployment.yaml
                    kubectl rollout status deployment/myapp --timeout=120s
                '''
            }
        }
    }

    post {
        success { echo 'Pipeline completed successfully.' }
        failure { echo 'Pipeline failed. Check console output.' }
    }
}
```

B. deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  labels:
    app: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
```

```
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myapp:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

C. alert-rules.yaml

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: k8s-alert-rules
  namespace: monitoring
  labels:
    release: monitoring
spec:
  groups:
    - name: k8s-alerts
      rules:
        - alert: PodCrashLoop
          expr: increase(kube_pod_container_status_restarts_total[5m]) > 3
          for: 1m
          labels:
            severity: critical
          annotations:
            summary: "Pod {{ $labels.pod }} is crash looping"
            description: "Container {{ $labels.container }} in namespace {{
$labels.namespace }} restarted more than 3 times in 5 minutes."

        - alert: HighCPUUsage
          expr: 100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[2m]))
* 100) > 80
          for: 2m
          labels:
            severity: warning
          annotations:
            summary: "High CPU usage on {{ $labels.instance }}"
            description: "CPU usage has been above 80% for more than 2 minutes."

        - alert: NodeDown
          expr: up{job="node-exporter"} == 0
          for: 1m
          labels:
            severity: critical
          annotations:
            summary: "Node {{ $labels.instance }} is down"
            description: "Node exporter target has been unreachable for more than 1
minute."
```

D. Command Cheat Sheet

Table 7 — Frequently used commands

Purpose	Command
SSH into EC2	`ssh -i devops-key.pem ubuntu@<EC2_PUBLIC_IP>`
Check Docker	`docker --version && docker ps`
Jenkins status	`sudo systemctl status jenkins`
Jenkins initial password	`sudo cat /var/lib/jenkins/secrets/initialAdminPassword`
Start Minikube	`minikube start --driver=docker`
Minikube status / IP	`minikube status` / `minikube ip`
List nodes	`kubectl get nodes`
List pods (all namespaces)	`kubectl get pods -A`
List services	`kubectl get svc`
Apply manifest	`kubectl apply -f <file>.yaml`
View pod logs	`kubectl logs <pod-name>`
Describe a resource	`kubectl describe pod <pod-name>`
Monitoring pods	`kubectl get pods -n monitoring`
List alert rules	`kubectl get prometheusrule -n monitoring`
Grafana password	`kubectl get secret -n monitoring monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 --decode`
Port-forward Grafana	`kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80 --address 0.0.0.0 &`
Port-forward Prometheus	`kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090 --address 0.0.0.0 &`
Port-forward Alertmanager	`kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-alertmanager 9093:9093 --address 0.0.0.0 &`
Find port-forwards	`ps aux | grep port-forward`
Stop all port-forwards	`pkill -f "kubectl port-forward"`
Helm releases	`helm list -A`
Simulate crash loop	`kubectl run crash-test --image=busybox --restart=Always -- /bin/sh -c "exit 1"`
Clean up test pod	`kubectl delete pod crash-test`